



DOCKER

Die Grundlagen der Containerisierung

Das Problem, das Docker löst

Von „Läuft bei mir“ zu „Läuft überall“



Ohne Docker

- Unterschiedliche Umgebungen auf Dev-, Test- und Produktionssystemen
- Abhängigkeitskonflikte zwischen Projekten (Python-, Node-Versionen etc.)
- Aufwendiges, fehleranfälliges manuelles Setup neuer Systeme



Mit Docker

- Anwendung + alle Abhängigkeiten in einem portablen Container
- Identisches Verhalten auf jedem System mit Docker
- Container starten in Sekunden statt Minuten

Die vier Kernkonzepte

Image, Container, Dockerfile und Registry greifen ineinander



Image

Unveränderliche Vorlage mit allem, was die Anwendung braucht – Code, Bibliotheken, Konfiguration.



Container

Eine laufende, isolierte Instanz eines Images. Mehrere Container können aus einem Image entstehen.



Dockerfile

Textdatei mit Bauanleitung: FROM, RUN, COPY, CMD – daraus entsteht das Image.



Registry

Zentrale Ablage für Images, z. B. Docker Hub – zum Herunterladen und Teilen.

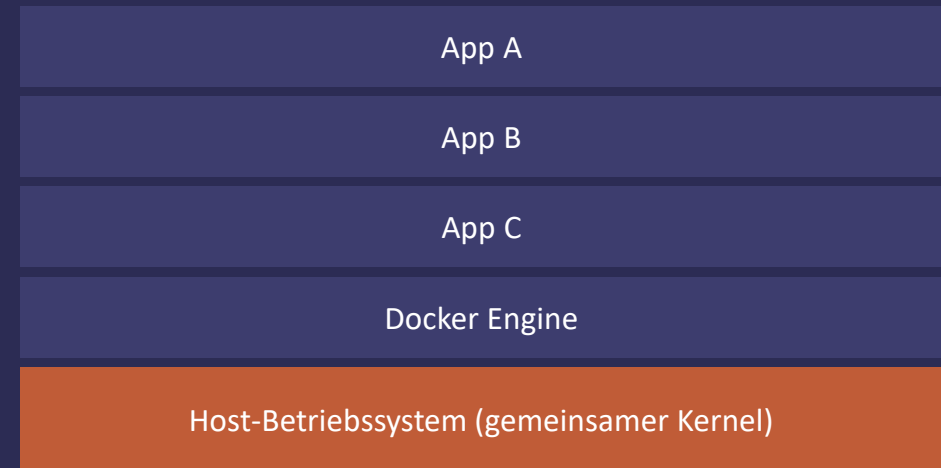
Wie es technisch funktioniert

Namespaces & cgroups statt eigenem Betriebssystem

Virtuelle Maschine

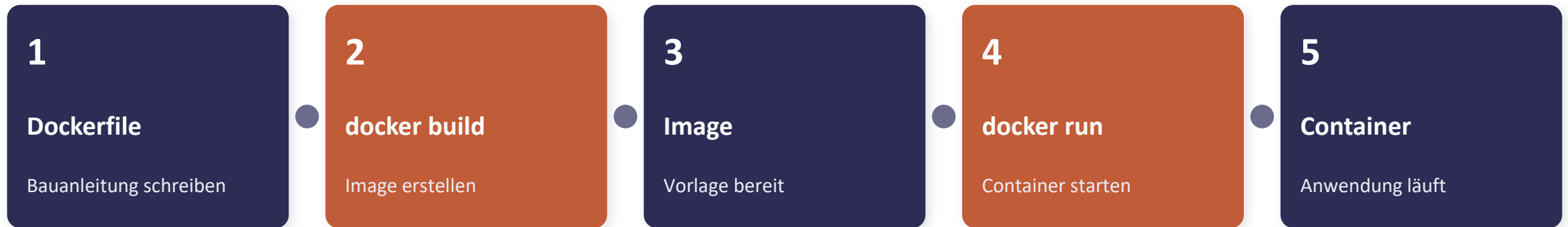


Docker Container



Der typische Workflow

Vom Dockerfile zum laufenden Container



Images landen in einer Registry (z. B. Docker Hub) und lassen sich von dort auf jedem System mit Docker abrufen und starten.

Warum Docker sich lohnt

Die wichtigsten Vorteile auf einen Blick



Konsistenz

Gleiche Umgebung auf Dev-Rechner, CI-Server und in Produktion.



Geschwindigkeit

Container starten in Sekunden – Megabyte statt Gigabyte.



Skalierbarkeit

Einfaches Verteilen und Vervielfältigen von Anwendungen.



Isolation

Verschiedene Versionen und Abhängigkeiten koexistieren konfliktfrei.



Danke für Ihre Aufmerksamkeit

Fragen zu Docker, Containerisierung oder unseren Kursangeboten?